



软件分析

数据流分析： 框架和扩展

熊英飞

北京大学



数据流分析框架



动机

- 上一节我们见到了四种具体的数据流分析
- 可以看出四种分析都有一个类似的形式
 - 能否统一放在一个框架中?
- 如何论证终止和合流?



数据流分析单调框架

- 数据流分析单调框架：对前面所述算法以及所有同类算法的一个通用框架
- 目标：通过配置框架的参数，可以导出各种类型的算法，并保证算法的安全性、终止性、合流性
- 为保证收敛性
 - 需要对抽象域的值加以限定
 - 需要对转换函数加以限定



半格 (semilattice)

- 半格是一个二元组 $(S, \sqcup)$ ，其中 S 是一个集合， $\sqcup : S \times S \rightarrow S$ 是一个合并运算，并且任意 $x, y, z \in S$ 都满足下列条件：
 - 幂等性idempotence: $x \sqcup x = x$
 - 交换性commutativity: $x \sqcup y = y \sqcup x$
 - 结合性associativity: $(x \sqcup y) \sqcup z = x \sqcup (y \sqcup z)$
- 有界半格是存在最小元 $\perp$ 的半格，满足
 - $x \sqcup \perp = x$



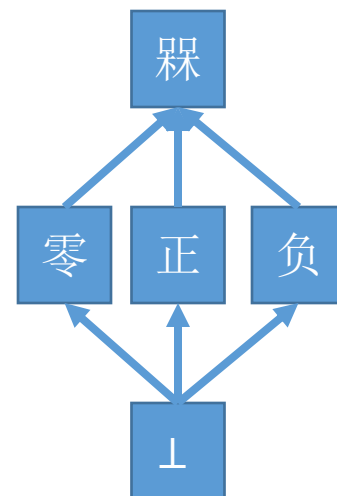
偏序 Partial Order

- 偏序是一个二元组 $(S, \sqsubseteq)$ ，其中 S 是一个集合， $\sqsubseteq$ 是一个定义在 S 上的二元关系，并且满足如下性质：
 - 自反性： $\forall a \in S: a \sqsubseteq a$
 - 传递性： $\forall x, y, z \in S: x \sqsubseteq y \wedge y \sqsubseteq z \Rightarrow x \sqsubseteq z$
 - 非对称性： $x \sqsubseteq y \wedge y \sqsubseteq x \Rightarrow x = y$
- 每个半格都定义了一个偏序关系
 - $x \sqsubseteq y$ 当且仅当 $x \sqcup y = y$



有界半格示例

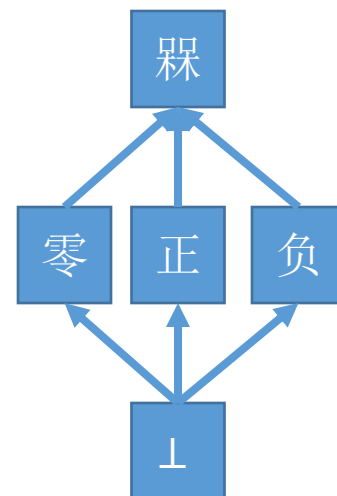
- 抽象符号域的五個元素和合并操作组成了一个有界半格
- 有界半格的笛卡尔乘积 $(S \times T, \sqcup_{xy})$ 还是有界半格
 - $(s_1, t_1) \sqcup_{xy} (s_2, t_2) = (s_1 \sqcup_x s_2, t_1 \sqcup_y t_2)$
- 任意集合的幂集和并集操作组成了一个有界半格
 - 偏序关系为子集关系
 - 最小元为空集
- 任意集合的幂集和交集操作组成了一个有界半格
 - 偏序关系为超集关系
 - 最小元为全集





半格的高度

- 半格的偏序图中任意两个节点的最大距离+1
- 示例：
 - 抽象符号域的半格高度为3
 - 集合和交集/并集组成的半格高度为集合大小+1
 - 活跃变量分析中半格高度为变量总数+1





练习

- 已知半格 $(S, \sqcup_S)$ 和半格 $(T, \sqcup_T)$ 的高度分别是 x 和 y , 求半格 $(S \times T, \sqcup_{ST})$ 的高度
 - $(s_1, t_1) \sqcup_{ST} (s_2, t_2) = (s_1 \sqcup_S s_2, t_1 \sqcup_T t_2)$
- 答案: $x+y-1$



单调（递增）函数

Monotone (Increasing) Function

- 给定一个偏序关系 $(S, \sqsubseteq)$ ，称一个定义在 S 上的函数 f 为单调函数，当且仅当对任意 $a, b \in S$ 满足
 - $a \sqsubseteq b \Rightarrow f(a) \sqsubseteq f(b)$
 - 注意：单调不等于 $a \sqsubseteq f(a)$
- 单调函数示例
 - 在符号分析的加减乘除操作均为为单调函数
 - 输入域为两个符号抽象域的笛卡尔乘积
 - 在集合和交/并操作构成的有界半格中，给定任意两个集合 $GEN, KILL$ ，函数 $f(S) = (S - KILL) \cup GEN$ 为单调函数



练习

- 以下函数是否是单调递增/递减的：
 - $f(x) = x - 1$
 - 定义域为实数，处处可导，且导数各处不为0的函数
 - 求集合 x 的补集
 - $f(x) = g \circ h(x)$ ，已知 g 和 h 是单调的
 - $f(x, y) = (g(x), h(y))$ ，已知 g 和 h 是单调的
 - 定义域看做由 (x, y) 组成的对
 - $f(x, y) = x \sqcup y$ ，已知 $x \in S, y \in S, (S, \sqcup)$ 是和偏序关系对应的有界半格
 - 定义域看做由 (x, y) 组成的对



数据流分析单调框架

- 一个控制流图 (V, E)
- 一个有限高度的有界半格 $(S, \sqcup, \perp)$
- 一个entry的初值 OUT_{entry}
- 一组单调函数，对任意 $v \in V - entry$ 存在一个单调函数 f_v
- 注意：对于逆向分析，变换控制流图方向再应用单调框架即可



数据流分析工单(WorkList) 算法

$\forall v \in (V - \text{entry}): \text{OUT}_v \leftarrow \perp$

$\text{ToVisit} \leftarrow V - \text{entry}$

While($\text{ToVisit.size} > 0$) {

$v \leftarrow \text{ToVisit}$ 中任意节点

$\text{ToVisit} -= v$

$\text{IN}_v \leftarrow \sqcup_{w \in \text{pred}(v)} \text{OUT}_w$

 If($\text{OUT}_v \neq f_v(\text{IN}_v)$) $\text{ToVisit} \cup= \text{succ}(v)$

$\text{OUT}_v \leftarrow f_v(\text{IN}_v)$

}



数据流分析收敛性

- 不动点：给定一个函数 $f: S \rightarrow S$ ，如果 $f(x) = x$ ，则称 x 是 f 的一个不动点
- 不动点定理：给定高度有限的有界半格 $(S, \sqcup, \sqcap)$ 和一个单调函数 f ，链 $\perp, f(\perp), f(f(\perp)), \dots$ 必定在有限步之内收敛于 f 的最小不动点，即存在非负整数 n ，使得 $f^n(\perp)$ 是 f 的最小不动点。
 - 证明：
 - 收敛于 f 的不动点
 - $\perp \sqsubseteq f(\perp)$ ，两边应用 f ，得 $f(\perp) \sqsubseteq f(f(\perp))$ ，
 - 应用 f ，可得 $f(f(\perp)) \sqsubseteq f(f(f(\perp)))$
 - 因此，原链是一个递减链。因为该格高度有限，所以必然存在某个位置前后元素相等，即，到达不动点。
 - 收敛于最小不动点
 - 假设有另一不动点 u ，则 $\perp \sqsubseteq u$ ，两边反复应用 f 可证



数据流分析收敛性

- 定义如下轮询函数

- $$F(\text{OUT}_{v_1}, \text{OUT}_{v_2}, \dots, \text{OUT}_{v_n}) =$$
$$\begin{aligned} & (f_{v_1}(\sqcup_{w \in \text{pred}(v_1)} \text{OUT}_w), \\ & f_{v_2}(\sqcup_{w \in \text{pred}(v_2)} \text{OUT}_w), \\ & \dots, \\ & f_{v_n}(\sqcup_{w \in \text{pred}(v_n)} \text{OUT}_w)) \end{aligned}$$

- 容易证明，F是单调函数

- 根据不动点定理，反复在 $(\perp, \dots, \perp)$ 上应用F所形成的链必然在有限步内终止，并且收敛于F的最小不动点



数据流分析收敛性

- 现在证明F和工单算法的结果等价
 - 二者主要的区别是工单每次随机选择ToVisit中的节点更新
- 终止性：
 - 令 OUT_v^i 为迭代第i轮之后的 OUT_v 值， v 为任意节点
 - 现在证明对任意节点 v ， $OUT_v^0, OUT_v^1, \dots$ 是一个递增序列，即每次增大或不变
 - 因为 $OUT_v^0 = \perp$ ，所以有 $OUT_v^0 \sqsubseteq OUT_v^1$
 - 假设到第k个元素都递增，现在证明 $OUT_v^k \sqsubseteq OUT_v^{k+1}$
 - 如果k+1轮没有更新v，则显然成立
 - 如果 $OUT_v^k = \perp$ ，则显然成立
 - 如果 $OUT_v^k \neq \perp$ ，则必然在某轮j被更新过。那么第j轮更新转换函数和合并操作的输入值都必然小于等于第k轮，同时因为转换函数和合并操作都是单调的，所以有 $OUT_v^k \sqsubseteq OUT_v^{k+1}$
 - 由于格的高度有限，所以对任意 v ， OUT_v 增大次数有限
 - ToVisit集合只在结果变化的时候才增加，否则减少，所以给定足够长的轮数，必然变为空集



数据流分析收敛性

- 合流性:
 - 令 X_i 为工单算法第 i 轮的计算结果
($OUT_{v_1}^i, OUT_{v_2}^i, \dots, OUT_{v_n}^i$)
 - 令 Y_i 为在 $(I, \perp, \dots, \perp)$ 上反复应用 F 的序列
 - 现在证明对任意 i , 有 $X_i \sqsubseteq Y_i$
 - $X_0 \sqsubseteq Y_0$
 - 假设 $X_k \sqsubseteq Y_k$, 因为工单只是更新一部分节点值, F 对所有节点值进行更新, 根据上一页的分析, 所以 $X_{k+1} \sqsubseteq F(X_k) \sqsubseteq F(Y_k)$
 - 因此, 当工单算法最终收敛的时候, 收敛的结果 $\sqsubseteq F$ 的最小不动点
 - 但由于工单算法收敛的结果也是 F 的不动点, 所以 工单算法收敛结果 = F 的最小不动点



数据流分析的安全性

- 数据流分析的输出值满足如下等式
$$\text{OUT}_v = f_v(\sqcup_{w \in \text{pred}(v)} \text{OUT}_w)$$
- 如果 f_v 保证单步转换的安全性， $\sqcup$ 保证合并的安全性，则分析整体安全
- 以上两者的安全性论证方式将之后结合抽象解释理论介绍



数据流分析的分配性

- 一个数据流分析满足分配性，如果
 - $\forall v \in V, x, y \in S: f_v(x) \sqcup f_v(y) = f_v(x \sqcup y)$
- 即：近似方案4（提前合并）不会引入不精确性
- 例：符号分析中的结点转换函数不满足分配性
 - 为什么？
 - 令 f_v 等于“加1”， $f_v(\text{正}) \sqcup f_v(\text{零})$



数据流分析的分配性

- 例：在集合和交/并操作构成的有界半格中，给定任意两个集合GEN, KILL，函数 $f(OUT) = (OUT - KILL) \cup GEN$ 满足分配性
 - $f(x) \cup f(y) = (x - K) \cup G \cup (y - K) \cup G = (x - K) \cup (y - K) \cup G = (x \cup y - K) \cup G = f(x \cup y)$
 - $f(x) \cap f(y) = ((x - K) \cup G) \cap ((y - K) \cup G) = ((x - K) \cap (y - K)) \cup G = (x \cap y - K) \cup G = f(x \cap y)$



复习：设计数据流分析

- 近似方案1：抽象状态代表程序的多个具体执行
 - 设计抽象域，对应的 α 、 γ 函数和初始值
- 近似方案2：针对控制流节点编写转换函数
 - 设计从基本语句导出转换函数的方法
- 近似方案3：在控制流路径分叉时，复制抽象状态到所有分支
 - 设计从条件导出压缩函数的方法（之后介绍）
- 近似方案4：在控制流路径合并时，用 $\sqcup$ 操作合并多个抽象状态
 - 设计 $\sqcup$ 操作



设计数据流分析（细化版）

- 近似方案1：抽象状态代表程序的多个具体状态
 - 设计抽象状态（=节点附加值）集合和入口初值
 - 需要和 $\sqcup$ 操作构成高度有限的半格
 - 需要存在保证分析正确性的 α 、 γ 函数
- 近似方案2：针对控制流节点编写转换函数
 - 设计从基本语句导出转换函数的方法
 - 需要保证转换函数为单调函数
 - 需要保证分析正确性
- 近似方案3：在控制流路径分叉时，复制抽象状态到所有分支
 - 设计从条件导出压缩函数的方法（之后介绍）
- 近似方案4：在控制流路径合并时，用 $\sqcup$ 操作合并多个抽象状态
 - 设计 $\sqcup$ 操作
 - 需要和抽象状态集合构成高度有限的半格



如何设计节点转换函数?

- 节点代码可能包含复杂表达式 $x := x + 1 - y$
- 如何从节点代码得到转换函数?
- 方法1: 考虑表达式求值的语义, 对应定义抽象语义并证明安全性
 - $Eval[e_1 + e_2](m) = Eval[e_1](m) + Eval[e_2](m)$
 - 符号分析 $[e_1 + e_2](\theta) = \text{符号分析}[e_1](\theta) \oplus \text{符号分析}[e_2](\theta)$
 - 可用表达式 $[e_1 + e_2] = \{e_1 + e_2\} \cup \text{可用表达式}[e_1] \cup \text{可用表达式}[e_2]$
- 方法2: 转成三地址码, 只处理每一条指令

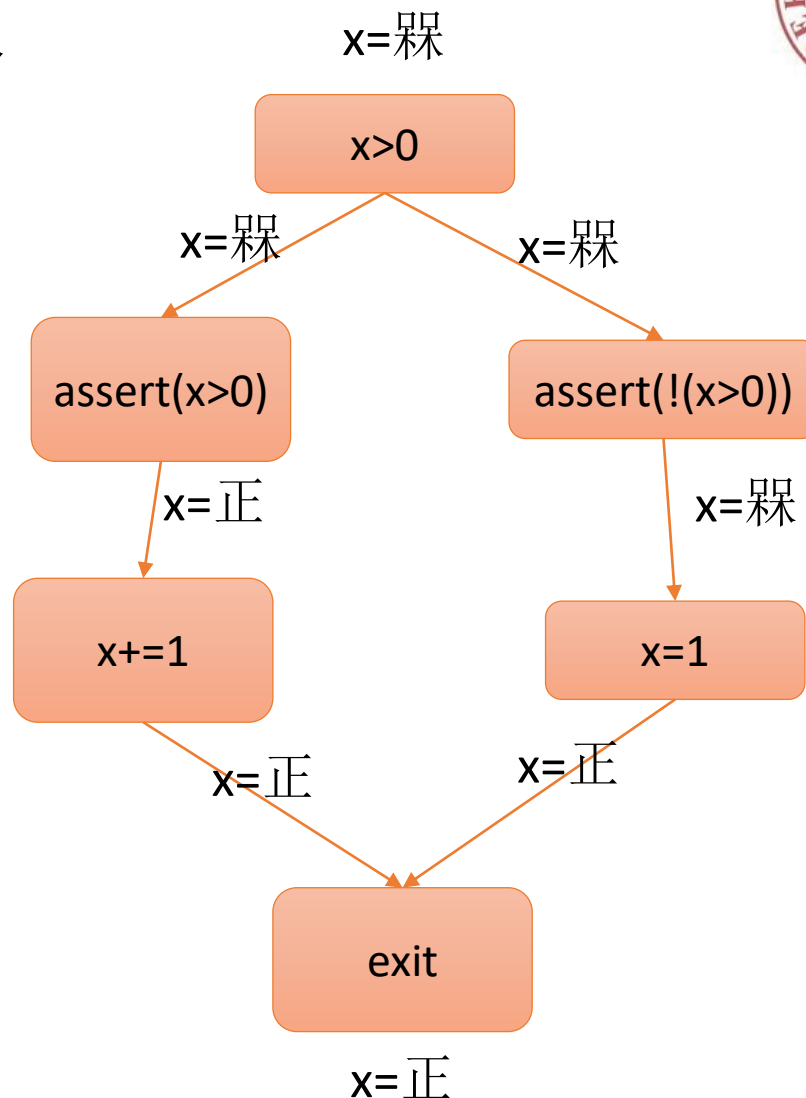


扩展：条件压缩函数



条件压缩函数

- 近似方案3：在控制流路径分叉时，复制抽象状态到所有分支
 - 每个具体状态只能到达一个分支，形成不精确
- 在每个分支添加条件压缩函数节点，根据条件压缩抽象值





如何设计条件压缩函数?

- 设计反向执行语义：给定输出的抽象值，计算输入的抽象值
 - 整数采用符号抽象，布尔值采用 $\{\perp, \text{真}, \text{假}, \text{值}\}$ ，其中 $\gamma(\text{值}) = \{\text{true}, \text{false}\}$
 - 反向 $[\wedge](\perp) = (\perp, \perp)$
 - 反向 $[\wedge](\text{真}) = (\text{真}, \text{真})$
 - 反向 $[\wedge](\text{假}) = (\text{值}, \text{值})$
 - 反向 $[\wedge](\text{值}) = (\text{值}, \text{值})$
 - 反向 $[> 0](\perp) = (\perp)$
 - 反向 $[> 0](\text{真}) = (\text{正})$
 - 反向 $[> 0](\text{其他}) = (\text{躲})$
 - 反向 $[*](\perp) = (\perp, \perp)$
 - 反向 $[*](\text{其他}) = (\text{躲}, \text{躲})$
 - 需要在抽象域上定义求交操作



如何设计条件压缩函数?

- 定义求交操作
 - $\text{甲} \sqcap \text{乙} = \alpha(\gamma(\text{甲}) \cap \gamma(\text{乙}))$
 - 例: $\text{赅} \sqcap \text{正} = \text{正}$, $\text{正} \sqcap \text{负} = \perp$
- 根据反向执行语义沿表达式从根到叶计算输入的抽象值, 然后和原来输入的值求交。
 - 如果一个变量出现多次的, 所有出现的值都要求交



基本反向语义：真分支

条件 $b = (x > 0) \wedge (y > 0)$

输入 $S = \{x \mapsto \text{真}, y \mapsto \text{真}\}$, 要求 b 为真。

① 反向 $[\wedge]$ (真) = (真, 真)

两个比较式都必须为真。

② 左侧：反向 $[> 0]$ (真) = 正, 得到 x 的约束“正”。

③ 右侧：反向 $[> 0]$ (真) = 正, 得到 y 的约束“正”。

④ 与输入求交： $x = \text{真} \sqcap \text{正}$, $y = \text{真} \sqcap \text{正}$ 。

输出 $S' = \{x \mapsto \text{正}, y \mapsto \text{正}\}$ 。



基本反向语义：假分支

条件 $b = (x > 0) \wedge (y > 0)$

输入 $S = \{x \mapsto \text{真}, y \mapsto \text{真}\}$, 要求 b 为假。

① 反向 $[\wedge](\text{假}) = (\text{值}, \text{值})$

至少一项为假，但无法单独确定哪一项。

② 两侧都应用反向 $[> 0](\text{值}) = \text{真}$ 。

与输入求交后， $S' = \{x \mapsto \text{真}, y \mapsto \text{真}\}$ 。

条件压缩函数并不总能压缩



基本反向语义：乘法

条件 $b = (x * y > 0)$

输入 $S = \{x \mapsto \text{正}, y \mapsto \text{负}\}$ ，要求 b 为真。

① 反向[> 0](真) = 正，因此要求 $x * y$ 为正。

② 基本规则：反向[*](正) = (负, 负)。

只看乘积，正 $\times$ 正、负 $\times$ 负都可能。

③ 与输入求交，得到 $S' = \{x \mapsto \text{正}, y \mapsto \text{负}\}$ 。

输入中已知“ x 为正”，可以推出“ y 为负”。关键是如何利用这个信息。



更精确的反向执行语义

- 参考输入的反向执行语义：给定输入输出的抽象值，压缩输入的抽象值
 - 反向[*](甲, 乙, $\perp$) = ($\perp$, $\perp$)
 - 反向[*](正, 甲, 正) = (正, 甲 $\sqcap$ 正)
 - 反向[*](负, 甲, 正) = (负, 甲 $\sqcap$ 负)
 - 反向[*](零, 甲, 正) = ($\perp$, $\perp$)
 - 反向[*](躲, 正, 正) = (正, 正)
 -
 - 反向[>](甲, 乙, $\perp$) = ($\perp$, $\perp$)
 - 反向[>](负, 甲, 真) = (负, 甲 $\sqcap$ 负)
 - 反向[>](零, 甲, 真) = (零, 甲 $\sqcap$ 负)
 -
- 首先采用正向语义计算出表达式的值，然后再用反向语义压缩变量的值



参考输入的反向语义：乘法

仍取 $b = (x * y > 0)$, $S = \{x \mapsto \text{正}, y \mapsto \text{赅}\}$ 。

① 正向：x 为正，y 为赅， $x * y$ 为赅，b 为值。

保存每个子表达式的正向结果。

② 反向：要求 b 为真， $x * y$ 为正。

③ 反向[*](正, 赅, 正) = (正, 赅 $\cap$ 正)
= (正, 正)

④ 在变量处求交，得到 $S' = \{x \mapsto \text{正}, y \mapsto \text{正}\}$ 。

已知 $x > 0$ 时， $x * y > 0$ 当且仅当 $y > 0$ 。



正向反向迭代

- 反向压缩变量的值之后，再进行一次正向反向流程可能得到更精确的值
 - $x > 0 \wedge y > x \wedge z > y$
 - 假设一开始 x, y, z 的值都是罅
 - 第一轮得到 $\{x \rightarrow \text{正}, y \rightarrow \text{罅}, z \rightarrow \text{罅}\}$
 - 第二轮得到 $\{x \rightarrow \text{正}, y \rightarrow \text{正}, z \rightarrow \text{罅}\}$
 - 第三轮得到 $\{x \rightarrow \text{正}, y \rightarrow \text{正}, z \rightarrow \text{正}\}$

条件压缩函数迭代的收敛性



- 如果反向语义函数保持单调，并且确保压缩或保持输入值，同时半格高度有限，那么该迭代过程一定收敛。
- 证明方式同前



条件压缩函数迭代的正确性

条件压缩函数的正确性：

输入抽象状态中所有满足条件的具体状态，都保留在压缩结果中。

反向语义的正确性：

抽象输入中所有落在抽象输出范围内的具体输入，都保留在结果抽象输入中。

定理：如果反向语义满足正确性，反向语义的多次迭代结果也保证正确性。

证明：表达式树上做归纳。对于每一轮，正向语义保证了所有符合条件的具体输入计算过程中产生的值都被包括在每一步的抽象值中，反向语义也是。



作业:

- 整数采用区间抽象，布尔值采用{⊥, 真, 假, 值}
 - 整数是数学定义，无上下界
 - 区间抽象: $[a, b]$ 表示大于等于 a 小于等于 b 的整数集合，其中 a 和 b 为任意整数
- 请针对下列操作设计参考输入的反向抽象语义
 - 逻辑与
 - 逻辑非
 - 大于
 - 加法
- 要求尽可能精确
- 同时分析基于你设计的反向语义，对任意表达式是否能保证收敛
 - 表达式中可包含变量、常量和以上四种操作符



参考资料

- 《编译原理》 第9章
- Lecture Notes on Static Analysis
 - <https://cs.au.dk/~amoeller/spa/>